

Turning on the real voiceover

The build picks its speech engine automatically, in this order:

ORDER	ENGINE	TRIGGER	OUTPUT
1	Gemini TTS (AI Studio)	<code>GEMINI_API_KEY</code>	Publishable, muxed into the ads
2	Google Cloud TTS	<code>GOOGLE_TTS_KEY</code>	Publishable, muxed into the ads
3	espeak + MBROLA	nothing set	Guide track, stamped, separate file

Prefer Gemini. It takes a natural-language direction alongside the line, and the two concepts want opposite reads — a wry founder recounting what he found, against a flat voice reading out a measurement. Those directions live in `vo-style.json` and are the main quality lever here; the voice matters less than the direction given to it.

Why Google and not the others

Probed from this environment, 22 Sep 2026:

SERVICE	REACHABLE
Gemini / AI Studio API (<code>generativelanguage.googleapis.com</code>)	yes
Google Cloud TTS (<code>texttospeech.googleapis.com</code>)	yes
AI Studio web UI (<code>aistudio.google.com</code>)	no — and it is a UI, not an API
MiniMax (<code>api.minimax.io</code> , <code>api.minimaxi.chat</code>)	no
ElevenLabs	no
OpenAI	no
Cartesia · Deepgram · PlayHT	no

The blocked ones fail at the network layer, before any credential is checked, so a key for them changes nothing on its own. To use one, add its API host to the allowed hosts in this Claude Code environment's settings first.

Setup — Gemini (recommended)

An AI Studio **page URL is not a credential**. The key is a separate thing.

- Go to <https://aistudio.google.com/apikey> and create an API key on the project you want billed. Free tier covers this volume comfortably.
- Add it to this environment's variables as `GEMINI_API_KEY`. Environment variables are read at session start, so a session already running will not see it — start a new one.
- Confirm, then build:

```
python3 vo.py --models # what this key actually serves python3 vo.py # should print "engine:
gemini" NODE_PATH=/opt/node22/lib/node_modules node build.js

--models matters because preview model ids move. The default is gemini-3.1-flash-tts-preview ; if the
key does not serve it, the error says so and GEMINI_TTS_MODEL overrides it.
```

Setup — Cloud TTS (alternative)

1. Enable **Cloud Text-to-Speech API** on the project and create an API key.
 2. Set it as `GOOGLE_TTS_KEY`.
 3. `python3 vo.py --voices` lists en-GB, en-US and ar-* voices, then build as above.
- The four motion posts rebuild with narration inside them. The `*-guide-vo.mp4` files are deleted automatically, because a guide is only needed while the read is synthetic.

The direction controls duration, not just tone

Measured on the same line, same voice, same model:

DIRECTION	"EVERY ONE OF THEM IS RUNNING ADS RIGHT NOW."
"...unhurried..."	6.12s
"...normal conversational speed, no dramatic pauses, this is a 25-second social video..."	3.60s

Same words, **41% shorter**. Across the whole script that was the difference between a 49-second TikTok ad and a 32-second one — and under 30 seconds is where short-form cold traffic actually performs, so the first version was unusable regardless of how good it sounded.

The lesson for anyone editing `vo-style.json`: **state the format and the pace, not only the mood**. "Unhurried" is a reasonable instruction for tone and a bad one for a feed. Every style string here names the target length on purpose. After editing, check the totals `vo.py` prints before rebuilding.

Rate limits, measured

- On the free tier, `gemini-3.1-flash-tts-preview` allows **3 requests per minute**. The script narrates line by line, so a full run is 16 requests and takes about six minutes. Three things make that painless:
- **Throttling**. Calls are spaced to stay inside the limit rather than hammering it and backing off. `GEMINI_TTS_RPM` raises it on a paid key.
 - **Retry**. A 429 is caught, the delay Google names in the response is honoured, and the line retries — up to six times.
 - **A cache**. Each line is keyed on its text, style, voice, model and engine. A re-run only pays for lines that actually changed, so fixing one word in the script costs one request, not sixteen. `vo/.cache.json` holds it; delete it to force a full regenerate.

Measured on this key, 22 Sep 2026:

MODEL / API	FREE-TIER VERDICT
<code>gemini-2.5-flash-preview-tts</code>	Works. Quota covers a full 16-line run. The default.
<code>gemini-3.1-flash-tts-preview</code>	Caps around 10 requests a day . One script is 16, so it cannot finish a run on the free tier. Marginally nicer read; worth switching to on a paid key.
<code>gemini-2.5-pro-preview-tts</code>	Quota exhausted immediately.
Cloud TTS (<code>texttospeech.googleapis.com</code>)	Rejects API keys entirely — HTTP 401, "API keys are not supported by this API. Expected OAuth2 access token". Needs a service account, not an AI Studio key.

So on a free AI Studio key there is exactly one workable route, and it is the default. Enabling billing on the project removes the cap and opens 3.1.

A two-word probe is not a valid test. `gemini-2.5-flash-preview-tts` returned no audio for the text "Quota probe." and full audio for a real sentence. When checking whether a model works, send a real line.

Security note

The key is sent as an `x-goog-api-key` header, never as a URL query parameter, so it cannot leak through a log line, a proxy access log or a traceback. Keep it in the environment. `.gitignore` covers `.gemini_key`, `*.key` and `.env`; nothing in this repository contains a credential.

Knobs

VARIABLE	DEFAULT	NOTES
<code>GEMINI_API_KEY</code>	—	Preferred. <code>GOOGLE_AI_STUDIO_KEY</code> also works.
<code>GEMINI_TTS_MODEL</code>	<code>gemini-3.1-flash-tts-preview</code>	Verify with <code>--models</code> ; preview ids move.
<code>GEMINI_TTS_VOICE</code>	<code>Charon</code>	A prebuilt Gemini voice. If the name is wrong the API says so.
<code>GOOGLE_TTS_KEY</code>	—	Cloud TTS instead. <code>GOOGLE_API_KEY</code> also works.
<code>GOOGLE_TTS_VOICE</code>	<code>en-GB-Neural2-D</code>	Run <code>--voices</code> and pick from what the account actually serves rather than trusting this default.
<code>GOOGLE_TTS_RATE</code>	<code>0.96</code>	Slightly under natural pace; the ad's argument is measurement, so an unhurried read suits it.

Scene lengths are re-measured from whichever engine ran, so the picture re-cuts itself to the new read. Nothing needs timing by hand.

Two things a better voice does not fix

Concept A speaks in the first person — "I filled in the contact form on N Dubai businesses". The campaign's whole argument is that the claim is literally true and independently checkable. A synthetic voice reading it is a

small dishonesty at the exact point the campaign asks to be trusted, and a good synthetic voice makes that worse rather than better, because nobody can tell. Recommend the founder records it: sixty seconds of phone audio. Concept B makes no first-person claim, so synthetic narration is fine there.

Arabic still needs a human. Google serves `ar-XA` voices and will read the Snapchat copy cleanly, but reading a draft well is not the same as the draft being right. That copy still needs a native Gulf-dialect writer before it runs.